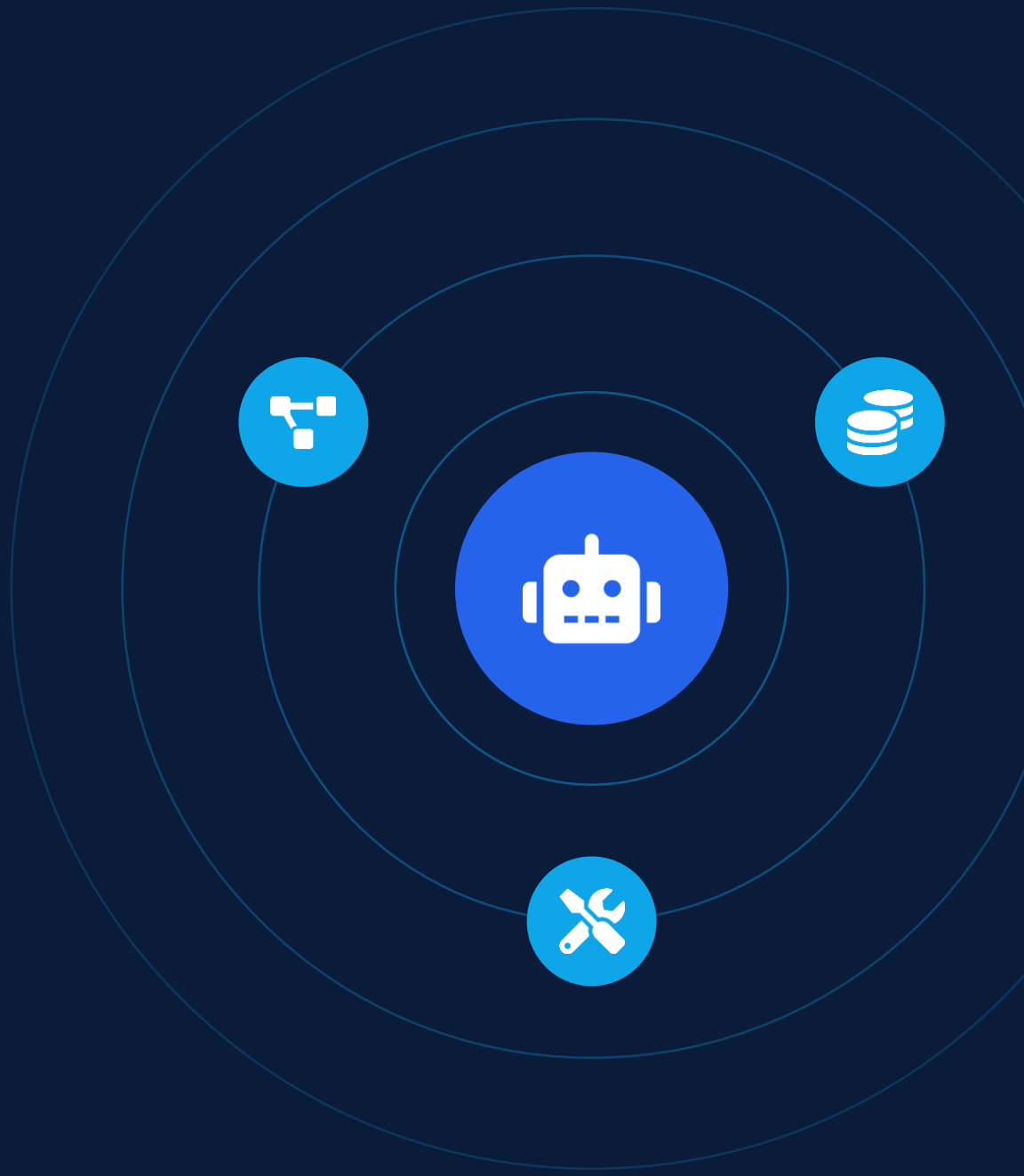
CONF42 · PROMPT ENGINEERING 2026

Observing AWS AgentCore with Datadog

End-to-End Observability for Autonomous AI



Indika Wimalasuriya

Director, Virtusa | AWS Community Builder | Datadog Champion | DevOps Institute Ambassador

AGENDA

What we'll cover

1. **Why agents break monitoring**
   Autonomy changes what "healthy" means

2. **Reference architecture**
   AgentCore, Bedrock and tools, observed by Datadog

3. **One trace, one view**
   APM, LLM Observability, Infrastructure and Logs

4. **Operating at scale**
   Cost, guardrails, kill switches and SLOs

YOU WILL LEAVE WITH



A practical reference architecture



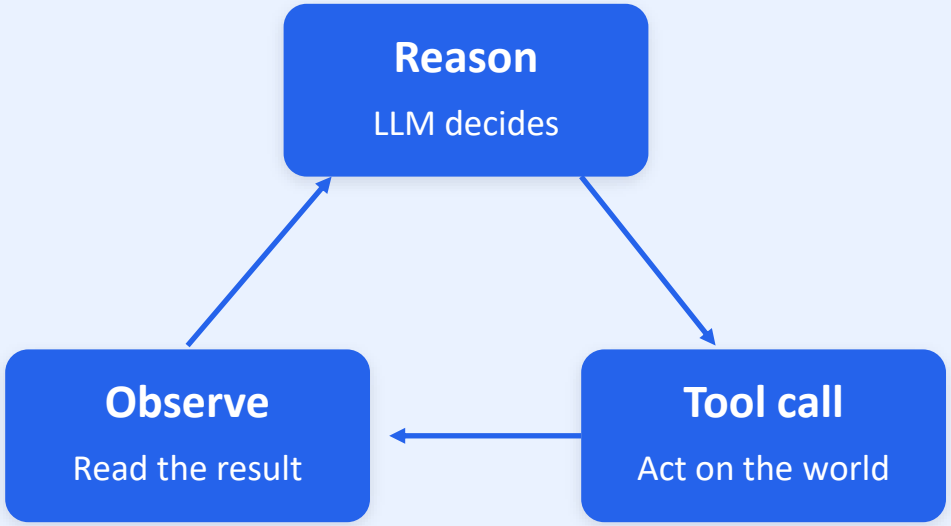
An SRE-oriented methodology

From deploying agents to operating them reliably at production scale

WHY

Agents don't just run, they act

THE AGENT LOOP



Non-deterministic: the step count varies on every run.

Is it running?
Availability and health of the agent runtime

What is it doing?
Reasoning steps and decisions per request

Which tools is it invoking?
Successes, failures and unexpected calls

How are models behaving?
Latency, errors, throttling and stop reasons

What does each interaction cost?
Tokens and spend per request and session

WHY

What breaks at 3 AM while dashboards stay green

Dashboard: ALL GREEN

- p95 latency OK
- Error rate 0.2%
- Lambda healthy
- CPU 34%

What's actually happening underneath



Agent looping indefinitely

Same tool, again and again



Hallucinated tool parameters

Valid call, wrong arguments



Context drift mid-session

Answers slowly leave the goal



$2K cost spike overnight

Tokens burn, no errors raised



Misaligned action taken

Succeeds, but wrongly

Healthy infrastructure is not the same as healthy agent behavior.

WHY

Six signals that make agent behavior observable



Agent decision trace

Why did the agent choose this next step?



Tool invocation spans

Every external call: latency, inputs, outputs



Token and cost telemetry

Prompt and completion tokens, cost per step



Model behavior signals

Stop reasons, errors, throttles, safety events



Session and memory state

What does the agent "remember" right now?

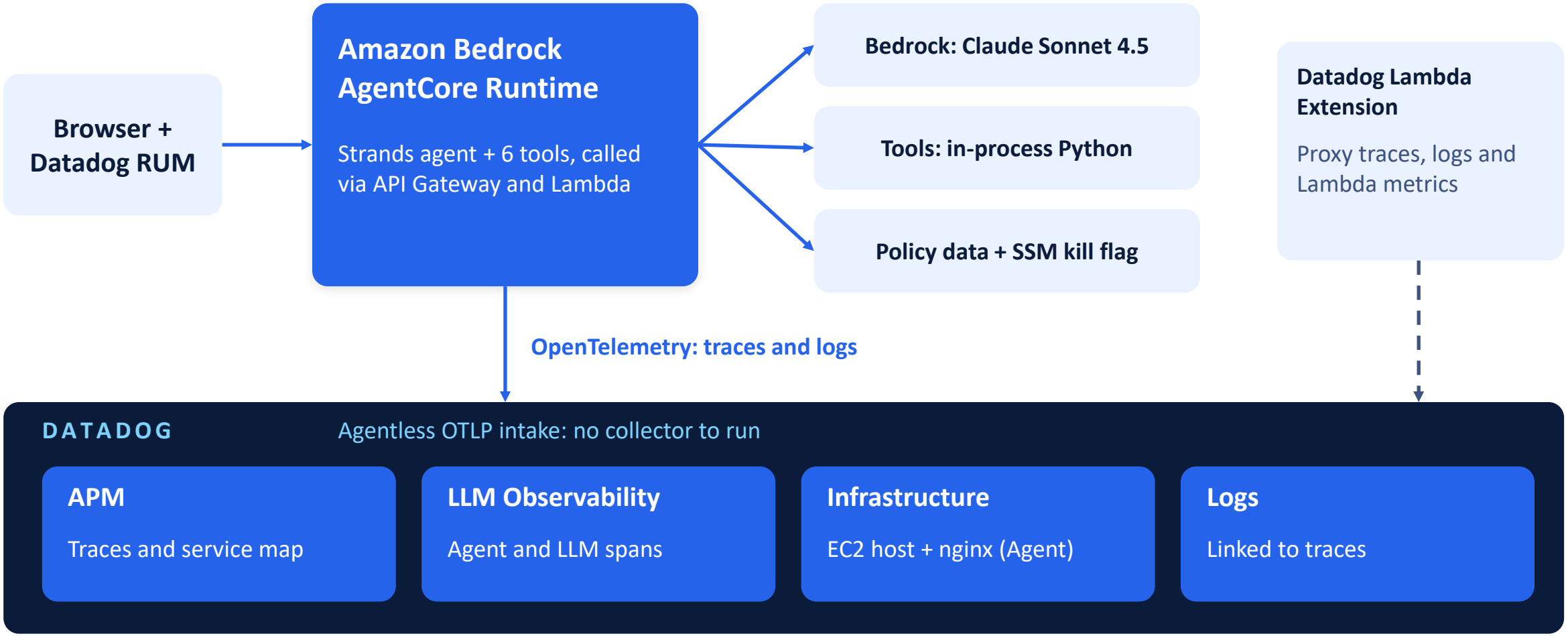


Guardrail evaluations

Did a safety check fire, and why?

HOW

Reference architecture: AgentCore + Datadog



HOW

Getting AgentCore telemetry into Datadog

1

Keep AgentCore OTel on

Leave ADOT running; add a Datadog exporter next to it in main.py.

2

Send OTLP to Datadog

Post spans to otlp.datadoghq.com with dd-otlp-source=llmobs. No collector.

3

Opt in to LLM Observability

Tag spans dd_llmobs_enabled=true and emit GenAI semconv v1.37+.

```
# AgentCore runtime environment
OTEL_SEMCONV_STABILITY_OPT_IN=gen_ai_latest_experimental
OTEL_TRACES_SAMPLER=always_on
DD_API_KEY=<datadog-api-key>
DD_SITE=datadoghq.com
```

Heads-up

Setting OTEL_EXPORTER_OTLP_* on AgentCore got 403s from Datadog. Without dd_llmobs_enabled, spans stay APM-only. The ml_app name comes from the root span service, not DD_LLMOBS_ML_APP.

HOW

Anatomy of an agent trace

SPAN AND KEY ATTRIBUTE	0 ms … 6,117 ms
invoke_agent Strands Agents `gen_ai.conversation.id = session`	6,117 ms
chat: LLM call 1 `2,508 in / 74 out tokens: tool_use`	1,949 ms
execute_tool estimate_repair_cost `status = service_unavailable`	30 ms: failed
chat: LLM call 2 `2,628 in / 74 out tokens: retry`	1,873 ms
execute_tool estimate_repair_cost `status = ok (retry succeeded)`	<1 ms: ok
chat: LLM call 3 `gen_ai.response.finish_reasons = end_turn`	2,258 ms

Agent span LLM call span Tool call span

Real trace from the demo

WHAT

One request, one operational view

APM

Where did the request spend its time?

Service map, latency and errors from Lambda proxy to agent

LLM Observability

What did the agent decide?

Agent, tool and LLM spans with prompts, tokens and cost

Infrastructure Monitoring

Is the platform healthy?

EC2 host, nginx and Lambda metrics, plus RUM sessions

Logs

What was the exact failure?

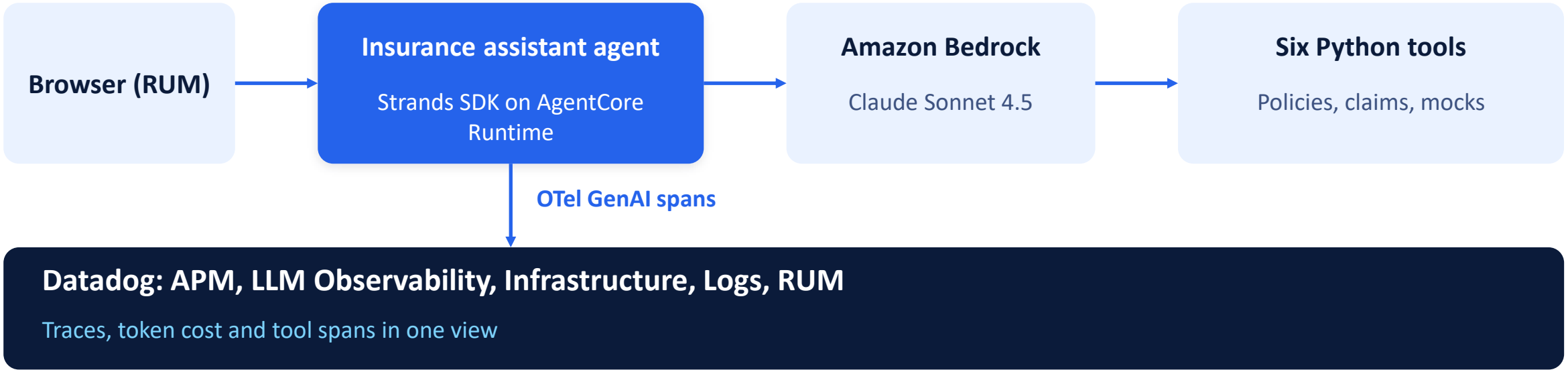
Application logs linked to the trace for instant context

Shared trace IDs and service, env and version tags tie all four views together

Pivot from an agent span to the Lambda span, the host and the log line without switching tools.

DEMO

Demo: tracing an insurance agent end to end



What to watch during the demo

1. One request expands into multi-step reasoning and tool calls
2. Parent and child spans across APM and LLM Observability
3. Token cost accumulating per session
4. Recovery behavior when a tool call fails

DEMO

Demo script: one prompt per signal

Prompt to type	What the agent does	What Datadog shows
$5,000 flood claim on HOME-STD-04	3 tool calls, then escalates to a human	guardrail.triggered tag
Repair cost for a dented bumper?	First tool call fails, agent quietly retries	Same tool span twice
Status of claim C-1213?	Loops about 5 times, then the kill switch trips	Tool spike, then suspended
$300 claim on policy ERR-500	Tool raises an exception, agent recovers	Red tool span, stack trace
What does motor cover? #outage	Whole request fails with HTTP 502	Errors in APM, logs, RUM



Escalations, retries and suspensions all stay green. Only ERR-500 and #outage raise real errors.

DEMO

What we learned wiring it up

Env vars alone got a 403

Setting OTEL_EXPORTER_OTLP_* on AgentCore failed; registering the exporter in code worked.

LLM Observability is opt-in

Without dd_llmobs_enabled=true on a span, OTel traces show in APM but never as agent runs.

Soft failures look green

Escalations and suspensions return a status, not an error, so nothing ever pages.

Loops burn tokens silently

The rogue loop raised no errors; only the tool-call rate and token count gave it away.

WHAT

Investigating failed and unexpected tool calls

1

Get the signal

A monitor on tool errors or error rate pages you with the affected service.

2

Open the trace

LLM Observability shows the agent run step by step, with inputs and outputs.

3

Find the tool span

Spot the failed or unexpected call, its arguments and its duration.

4

Pivot to the cause

Jump to the downstream APM span, infrastructure metrics and trace logs.

Failed call looks like

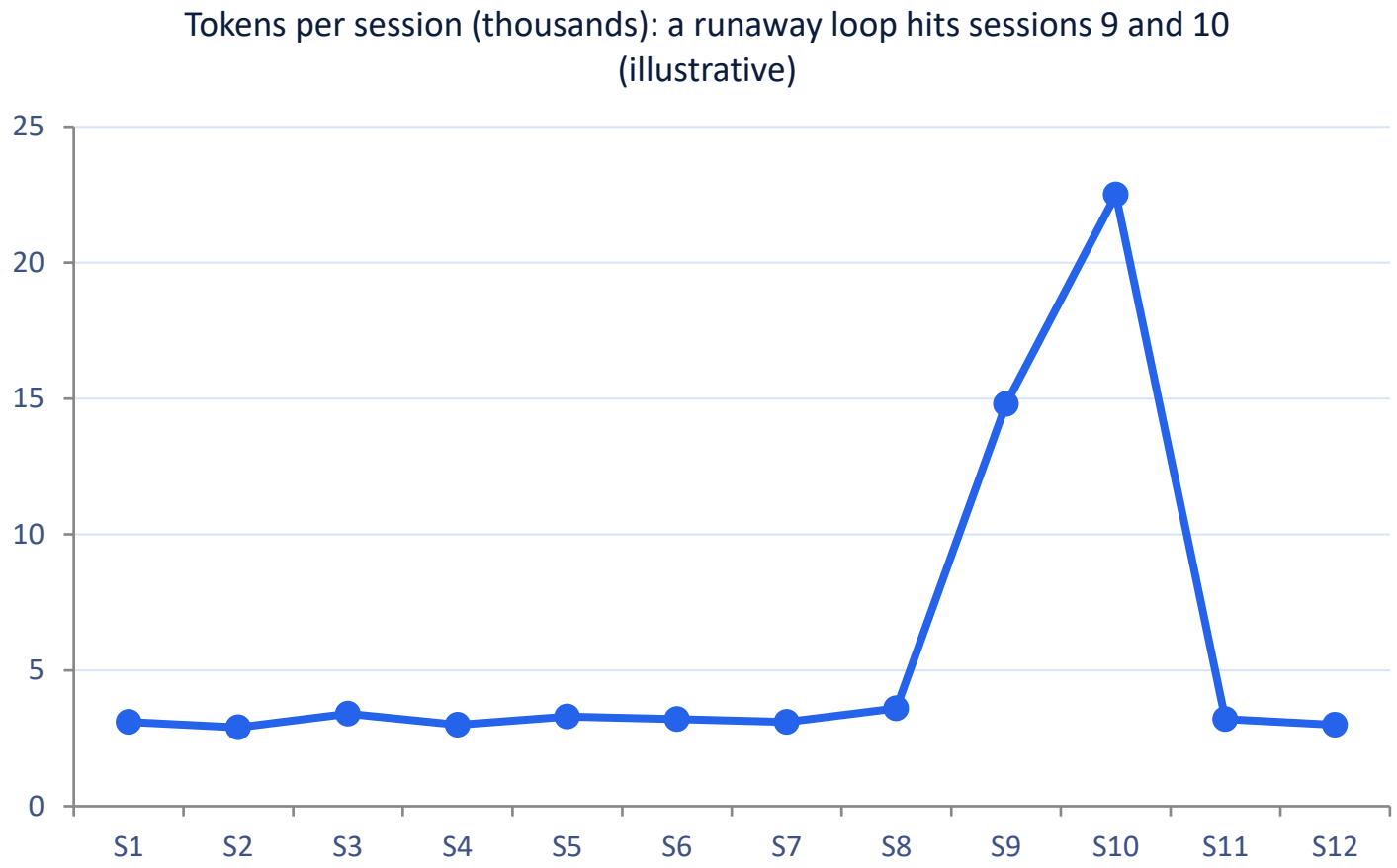
- Error status or exception on the tool span
- Timeouts and throttling from a downstream service
- Malformed output that triggers agent retries

Unexpected call looks like

- Wrong tool chosen for the intent of the user
- Hallucinated or out-of-range parameters
- Same tool called repeatedly: a loop forming

WHAT

Catching abnormal token and cost consumption



Track tokens per session

Input and output tokens by agent, model and tool

Attribute the spend

Tag by agent, team and environment to see who spends what

Alert on rate of change

Anomaly monitors catch loops long before the invoice does

WHAT

Operational guardrails for production agents

Agent running

Production traffic

Telemetry

OTel spans, metrics, logs

Datadog monitors

Thresholds and anomaly detection

WHAT

An SRE methodology for autonomous AI

Reliability question	Service level indicator	Example objective
Does the agent finish the job?	Task success rate: no error, guardrail passed	99% of sessions
Is it fast enough?	p95 end-to-end latency per session	p95 under 5 s
Do its tools behave?	Tool-call error rate and calls per session	Errors under 1%
Is spend under control?	Tokens and cost per session	Within 2x of baseline
Are guardrails working?	Guardrail trigger and block rate	Alert on deviation



Example targets: tune to your workload, and alert on error-budget burn rate rather than single spikes.

WHAT

Lessons from running agents in production

Instrument before you deploy

Retrofitting telemetry after an incident means flying blind exactly when you need data most.

Propagate trace context early

Multi-agent and tool hops only stitch into one trace if context travels with every call.

Observe the guardrails too

A misconfigured guardrail is invisible unless its decisions are recorded as telemetry.

Treat tokens as a production metric

Unmonitored token burn can break the business case long before it breaks the service.

WRAP-UP

Key takeaways

1. Agents fail in ways classic monitoring cannot see: loops, bad tool calls and cost spikes.
2. AgentCore emits OpenTelemetry, and Datadog correlates it with APM, Logs and Infrastructure.
3. LLM Observability turns each request into a readable trace of decisions, tools and tokens.
4. Operate with SLOs, anomaly monitors and a kill switch built before go-live.

Thank you

